

Module 5

Deep Learning (Deep Neural Networks)

Resources

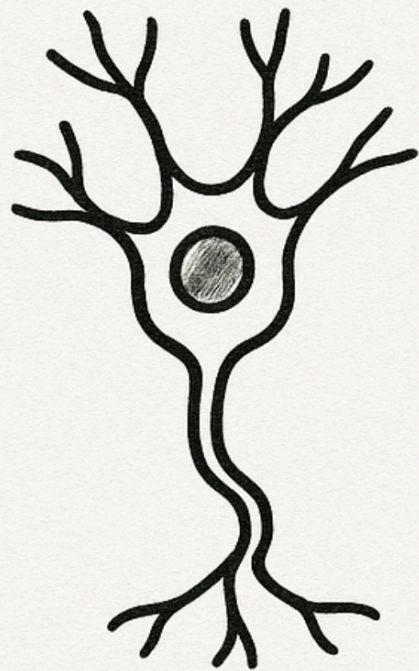
[PyTorch a 60-minute blitz](#)

Goodfellow et al. (2016) [Deep Learning](#)

Deep Neural Networks

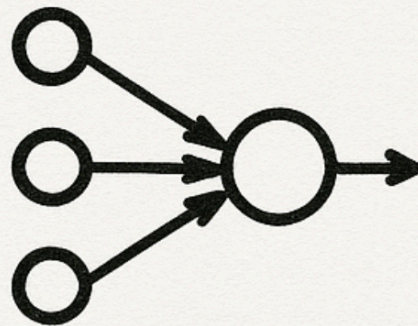
A very short history

A SHORT HISTORY OF NEURAL NETWORKS



1943

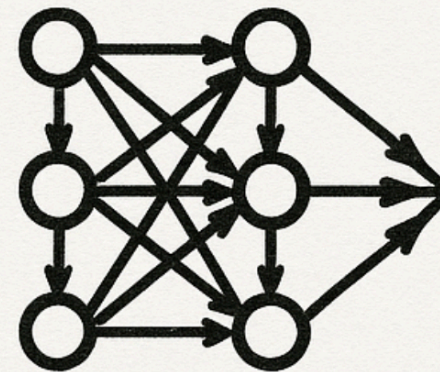
1943



McCULLOCH-
PITTS MODEL

1943

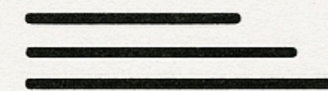
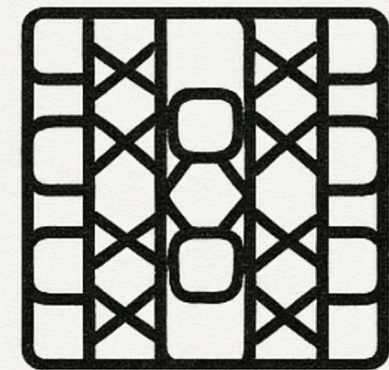
1980s



BACKPROP

2000s

2000s

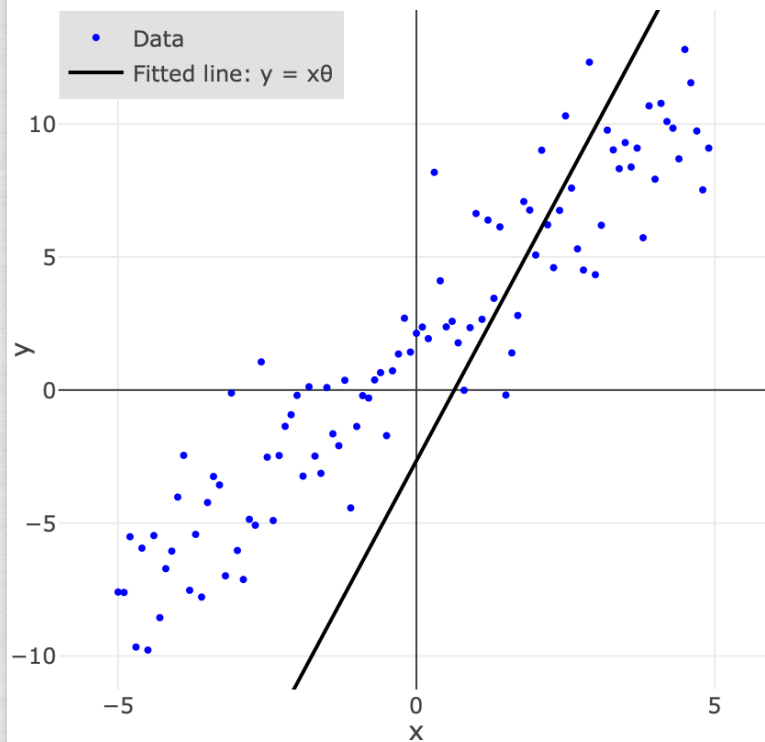


2010s

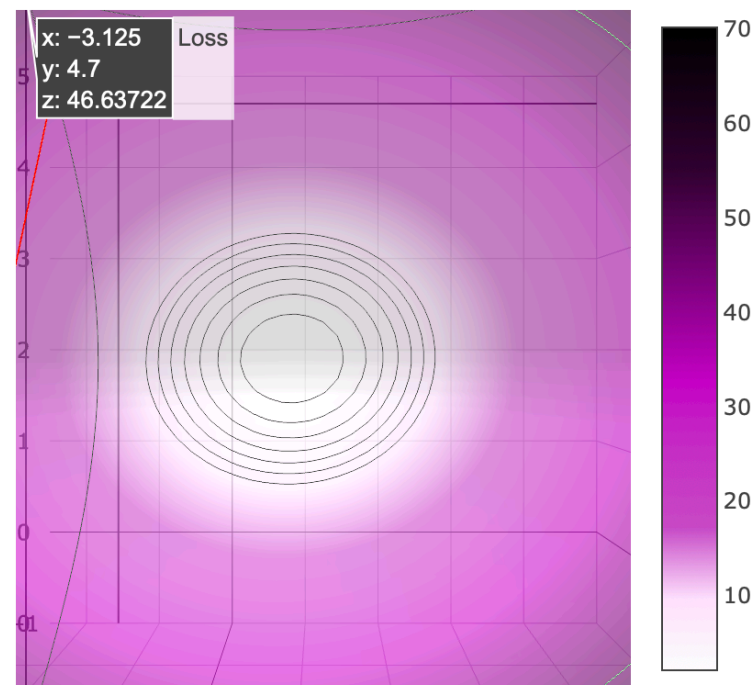
Learning through back-propagation

Gradient Descent

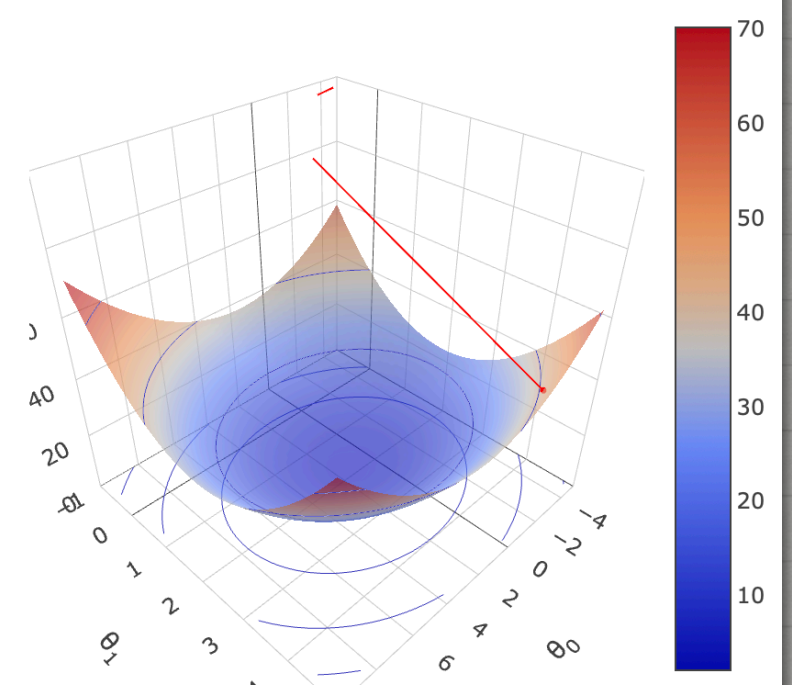
Labelled Data & Model Output



Contour Loss Function and Descent Path



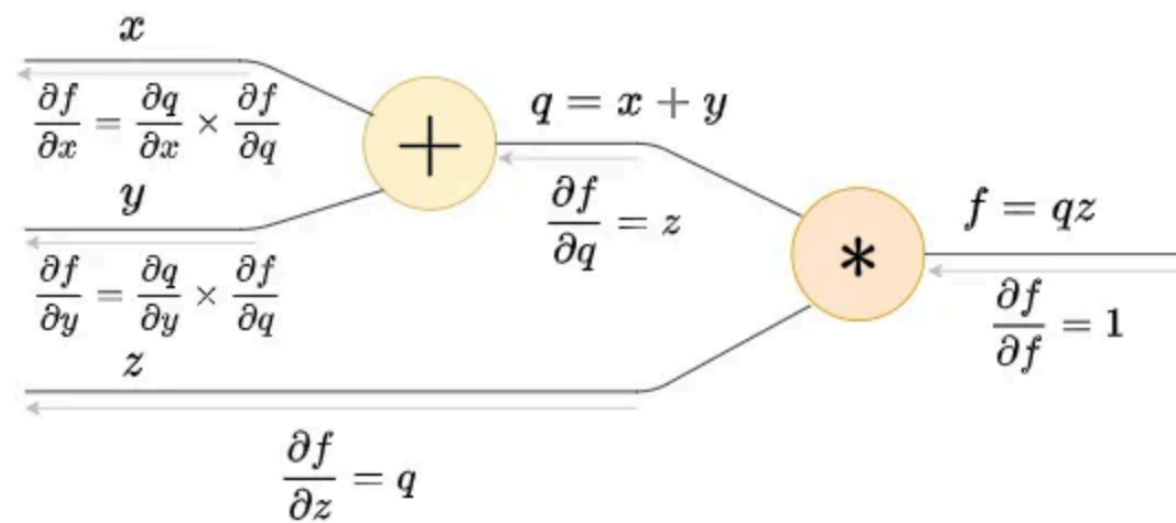
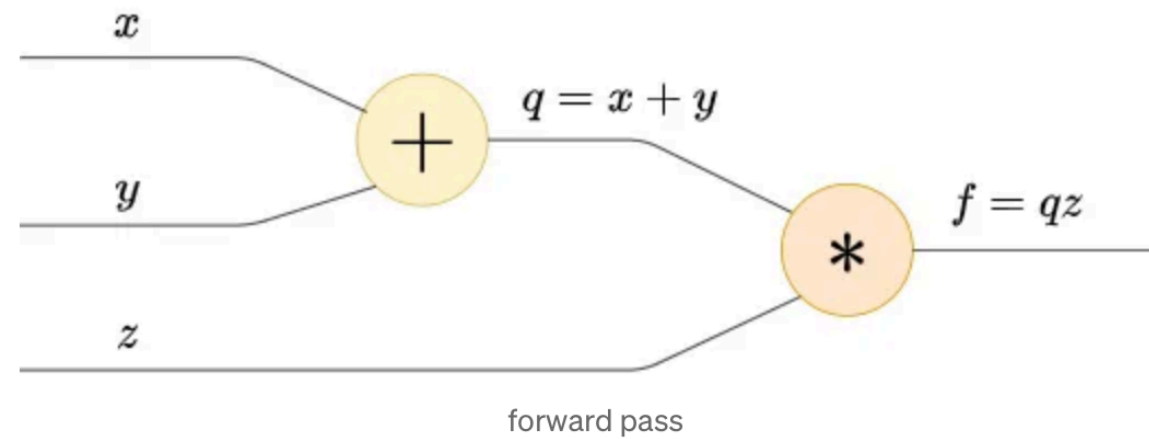
3D Loss Function and Descent Path



https://aero-learn.imperial.ac.uk/vis/Machine Learning/gradient_descent_3d.html

Learning through back-propagation

Credit assignment & backprop



The light gray arrows represent backward pass

Support for Deep Learning in Python

PyTorch:

```
$ conda activate cog-ai
```

```
$ pip install torch torchvision
```

Tensors (Like numpy arrays, but better):

```
x = torch.tensor([2.0, 3.0])
```

Autograd:

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
```

torch.nn:

A library containing different types of NN layers and operations on those layers:

e.g. nn.Linear, nn.Sequential, nn.ReLU, nn.Conv2D, etc.

DataLoader:

An iterable utility that wraps a Dataset to provide an easy and efficient way to access data in batches. It handles tasks like automatically grouping samples into batches, shuffling the data each epoch, and parallelizing the data loading process across multiple worker processes

Mini-project 4: A small Neural Network

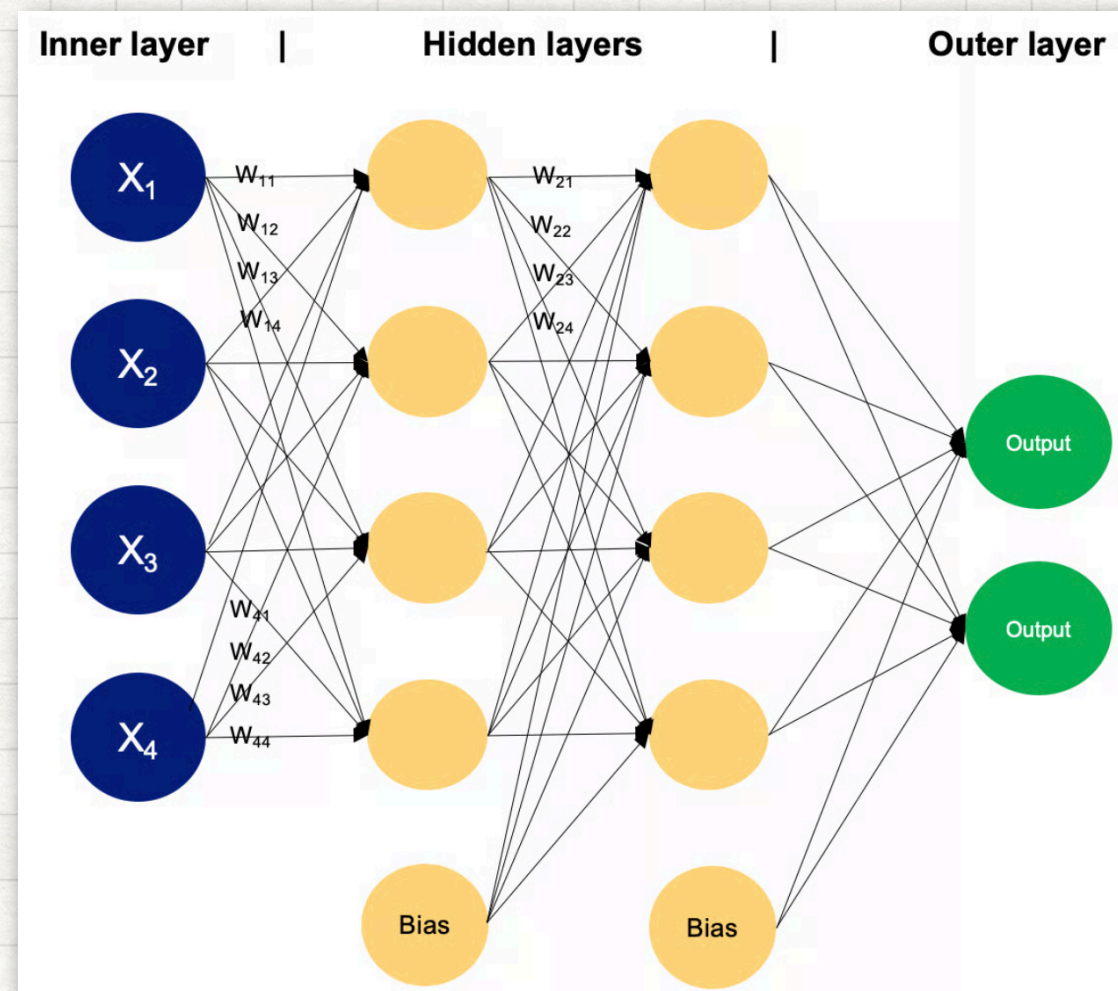
Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



MNIST Database

Mini-project 4: A small Neural Network

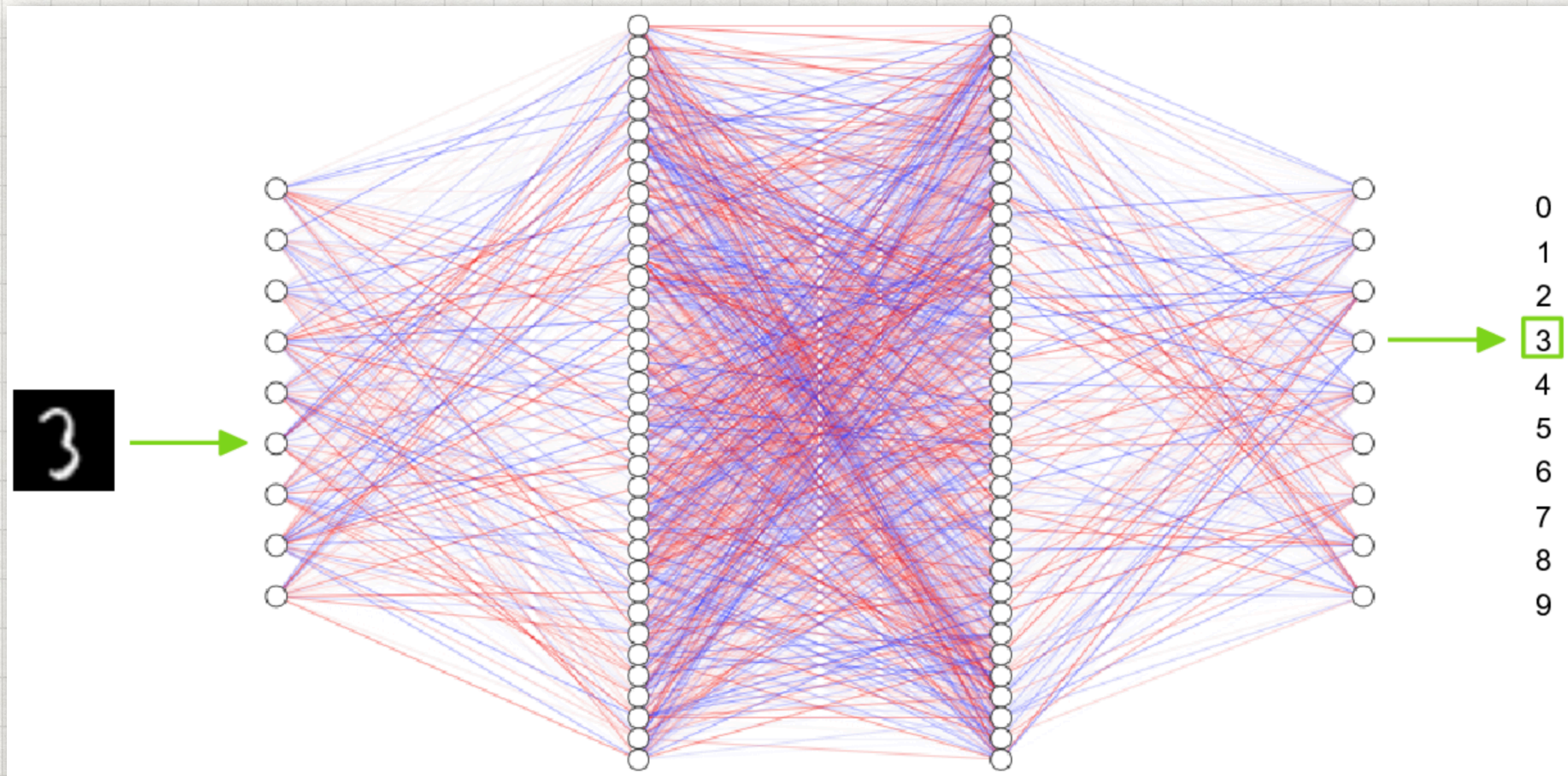
Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Multi-layer Perceptron

Mini-project 4: A small Neural Network

Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Demo 1: Using Tensors & Autograd

Define tensors 'a' and 'b'. Perform series of calculations on these tensors, using variables 'c', 'd', and 'e' to store outputs. Then print the gradient of 'e' with respect to 'a' and 'b'.

$$c = a + b$$

$$d = b + 1$$

$$e = c * d$$

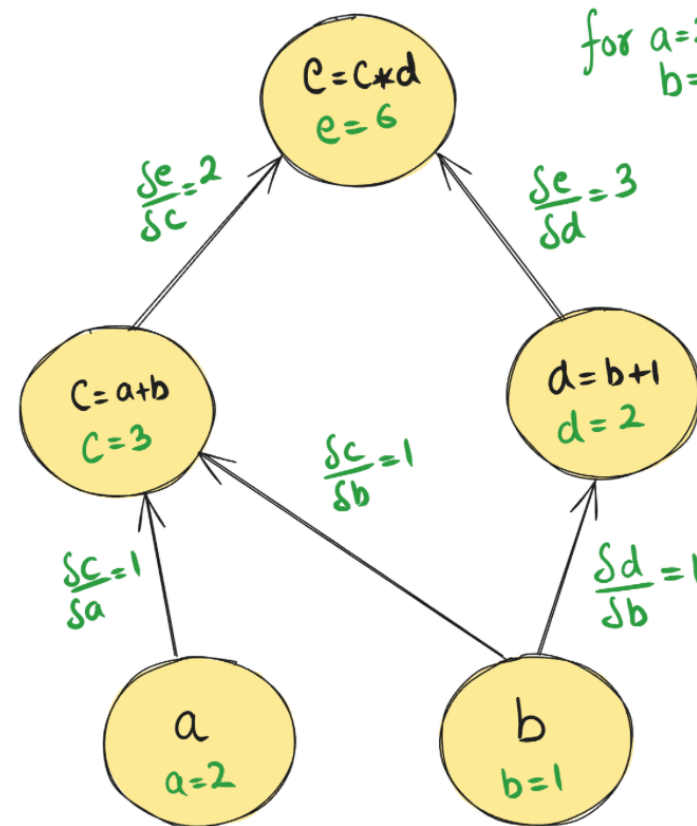
Compute: $\frac{\delta e}{\delta a}$ & $\frac{\delta e}{\delta b}$

Demo 1: Using Tensors & Autograd

Computational Graph

Computational Graph

Equation $e = (a * b)(b + 1)$ } can be written as
 $c = a + b$
 $d = b + 1$
 $e = c * d$



$$\frac{de}{da} = \frac{de}{dc} \times \frac{dc}{da} = 2 \times 1$$

$$\frac{de}{db} = \frac{de}{da} \times \frac{da}{db} + \frac{de}{dd} \times \frac{dd}{db} = 3 \times 1 + 2 \times 1 = 5$$